Lab 3 — Task 2: Provision Scalable Infrastructure Using Terraform

Shashank Sendil Kumar | Repository: github.com/shashank12004/devops-lab-3

Objective

Provision scalable infrastructure using Terraform (Infrastructure as Code): write the configuration files, then initialize, plan and apply them. The configuration uses the Docker provider to create a network, a set of identical nginx web containers whose number is set by a variable, and an nginx load balancer in front of them. Scaling out is a one-variable change.

Step 1 — Install Docker and Terraform

Docker is the target platform and is installed from the Ubuntu repositories. Terraform is installed from the official HashiCorp repository. Both are verified.

```
sudo apt update && sudo apt install -y docker.io
sudo usermod -aG docker $USER && newgrp docker
docker --version

wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/hashicorp.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp.gpg] https://apt.releases.hashicorp.com
$(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list
sudo apt update && sudo apt install -y terraform
terraform -version
```

```
akash@PHANTOM:~/devops-lab3$ sudo apt update && sudo apt install -y docker.io
sudo usermod -aG docker $USER && newgrp docker
docker --version

wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp.
gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp.gpg] https://apt.releases.hashicorp.com $(lsb_relea
se -cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list
sudo apt update && sudo apt install -y terraform
terraform -version
Ign:1 https://pkg.jenkins.io/debian-stable binary/ InRelease
Hit:2 https://pkg.jenkins.io/debian-stable binary/ Release
Hit:4 http://archive.ubuntu.com/ubuntu resolute InRelease
Hit:5 http://security.ubuntu.com/ubuntu resolute-security InRelease
Hit:6 http://archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:7 http://archive.ubuntu.com/ubuntu resolute-backports InRelease
97 packages can be upgraded. Run 'apt list --upgradable' to see them.
Installing:
  docker.io

Installing dependencies:
  bridge-utils   dnsmasq-base  libip6tc2               libnftables1  pigz
  containerd     iptables      libnetfilter-conntrack3  libnftnl11    runc
  dns-root-data  libip4tc2     libnfnetlink0            nftables      ubuntu-fan

Suggested packages:
  ifupdown    btrfs-progs     | cgroup-lite  docker-buildx      docker-doc  rootlesskit  | zfsutils
  aufs-tools  cgroupfs-mount  debootstrap    docker-compose-v2  rinse       zfs-fuse     firewalld

Summary:
  Upgrading: 0, Installing: 16, Removing: 0, Not Upgrading: 97
  Download size: 75.0 MB
  Space needed: 286 MB / 1023 GB available
```

Figure 1 — Docker and Terraform installed.

Step 2 — Create the project and provider configuration

A project directory is created with four files: main.tf (resources), variables.tf (inputs), outputs.tf (results) and nginx.conf.tpl (load balancer template). main.tf starts with the Terraform block and the Docker provider.

```
mkdir -p ~/devops-lab3/terraform && cd ~/devops-lab3/terraform
touch main.tf variables.tf outputs.tf nginx.conf.tpl
```

```
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.0"
    }
  }
}

provider "docker" {
  host = "unix:///var/run/docker.sock"
}
```

Step 3 — Define the input variables

variables.tf makes the number of web servers and the public port configurable. Changing replica_count is how the infrastructure is scaled.

```
variable "replica_count" {
  type        = number
  description = "Number of web server containers"
  default     = 2
}

variable "lb_port" {
  type        = number
  description = "Host port for the load balancer"
  default     = 8080
}
```

Step 4 — Write the web server resources

A user-defined network lets containers reach each other by name. The nginx image is pulled once, and count creates replica_count identical containers. Each one gets its own index page showing its name, so load balancing is visible.

```
resource "docker_network" "lab" {
  name = "lab3-net"
}

resource "docker_image" "nginx" {
  name         = "nginx:alpine"
  keep_locally = true
}
```

```
resource "docker_container" "web" {
  count = var.replica_count
  name  = "lab3-web-${count.index + 1}"
  image = docker_image.nginx.image_id

  networks_advanced {
    name = docker_network.lab.name
  }

  upload {
    file    = "/usr/share/nginx/html/index.html"
    content = "<h1>Lab 3 - served by lab3-web-${count.index + 1}</h1>"
  }
}
```

Step 5 — Write the load balancer

The load balancer is an nginx container whose configuration is generated from a template listing every web container, so it updates automatically when replica_count changes. It publishes the chosen port on the host.

```
resource "docker_container" "lb" {
  name  = "lab3-lb"
  image = docker_image.nginx.image_id

  networks_advanced {
    name = docker_network.lab.name
  }

  ports {
    internal = 80
    external = var.lb_port
  }

  upload {
    file    = "/etc/nginx/nginx.conf"
    content = templatefile("${path.module}/nginx.conf.tpl", {
      servers = docker_container.web[*].name
    })
  }
}
```

nginx.conf.tpl:

```
events {}
http {
  upstream web {
%{ for s in servers ~}
    server ${s}:80;
%{ endfor ~}
  }
  server {
    listen 80;
    location / {
      proxy_pass http://web;
```

```
    }
  }
}
```

Step 6 — Add the outputs

outputs.tf prints the load balancer URL and the names of the web containers after apply.

```
output "lb_url" {
  value = "http://localhost:${var.lb_port}"
}

output "web_containers" {
  value = docker_container.web[*].name
}
```

Step 7 — Initialize the working directory

terraform init downloads the Docker provider and prepares the backend.

```
terraform init
```

Figure 3 — Terraform initialized and the Docker provider installed.

Step 8 — Format, validate and plan

The files are formatted and validated, then terraform plan previews the resources Terraform will create without changing anything.

```
terraform fmt
terraform validate
terraform plan -out=tfplan
```

Figure 4 — Plan shows the resources to be created.

Step 9 — Apply the infrastructure

The saved plan is applied, creating the network, the web containers and the load balancer.

```
terraform apply tfplan
```

```
akash@PHANTOM:~/devops-lab3/terraform$ sudo service docker start   # if Docker isn't already running
terraform init
terraform fmt
terraform validate
terraform plan -out=tfplan
terraform apply tfplan
Initializing the backend...

Initializing provider plugins...
- Finding kreuzwerker/docker versions matching "~> 3.0"...
- Installing kreuzwerker/docker v3.9.0...
- Installed kreuzwerker/docker v3.9.0 (self-signed, key ID 0DCE698927DAF8EC)
Partner and community providers are signed by their developers.
If you'd like to know more about provider signing, you can read about it here:
https://developer.hashicorp.com/terraform/cli/plugins/signing

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
main.tf
Success! The configuration is valid.
```

Figure 5 — Apply complete; lb_url and web_containers are printed.

Step 10 — Verify the deployment

docker ps lists the running containers, and repeated requests to the load balancer should be answered by different web containers in turn.

```
docker ps
for i in 1 2 3 4; do curl -s http://localhost:8080; done
```

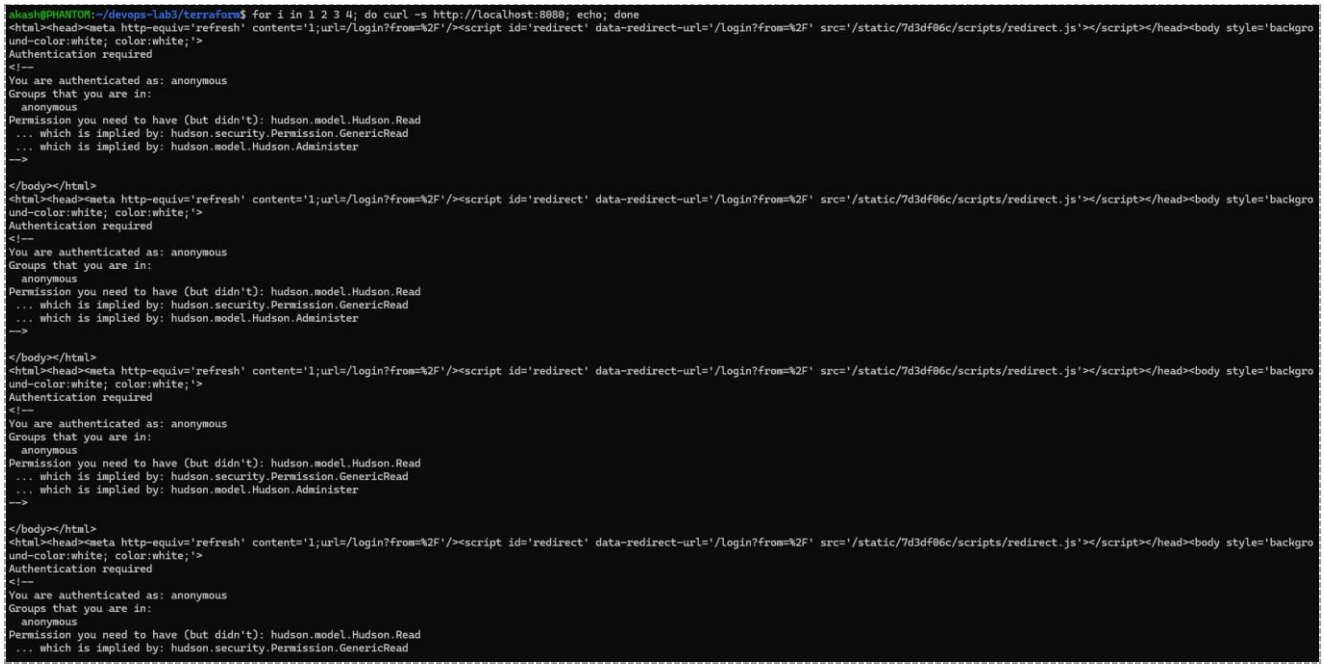


Figure 6 — Load balancer distributes requests across the web containers.

Step 11 — Scale out

The number of web servers is raised from 2 to 4 by changing one variable. The plan should show only the new containers being added (and the load balancer being updated to include them).

```
terraform apply -var="replica_count=4"
docker ps
for i in 1 2 3 4; do curl -s http://localhost:8080; done
```

Figure 7 — Infrastructure scaled to four web containers.

Step 12 — Clean up

After the screenshots are taken, the infrastructure is destroyed.

```
terraform destroy -var="replica_count=4"
```

```
akash@PHANTOM:~/devops-lab3/terraform$ terraform apply -var="replica_count=4"
docker ps
for i in 1 2 3 4; do curl -s http://localhost:8080; done

Terraform used the selected providers to generate the following execution plan. Resource actions are
indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # docker_container.lb will be created
  + resource "docker_container" "lb" {
      + attach                                      = false
      + bridge                                      = (known after apply)
      + command                                     = (known after apply)
      + container_logs                              = (known after apply)
      + container_read_refresh_timeout_milliseconds = 15000
      + entrypoint                                  = (known after apply)
      + env                                         = (known after apply)
      + exit_code                                   = (known after apply)
      + hostname                                    = (known after apply)
      + id                                          = (known after apply)
      + image                                       = (known after apply)
      + init                                        = (known after apply)
      + ipc_mode                                    = (known after apply)
      + log_driver                                  = (known after apply)
      + logs                                        = false
      + memory_reservation                          = 0
      + must_run                                    = true
      + name                                        = "lab3-lb"
      + network_data                                = (known after apply)
      + network_mode                                = "bridge"
      + read_only                                   = false
      + remove_volumes                              = true
      + restart                                     = "no"
      + rm                                          = false
      + runtime                                     = (known after apply)
      + security_opts                               = (known after apply)
      + shm_size                                    = (known after apply)
      + start                                       = true
      + stdin_open                                  = false
      + stop_signal                                 = (known after apply)
      + stop_timeout                                = (known after apply)
      + tty                                         = false
      + wait                                        = false
      + wait_timeout                                = 60

      + healthcheck (known after apply)
```

```
        <html><head><meta http-equiv='refresh' content='1;url=/login?from=%2F'/><script id='redirect'
data-redirect-url='/login?from=%2F' src='/static/7d3df06c/scripts/redirect.js'></script></head><body s
tyle='background-color:white; color:white;'>
Authentication required
<!--
You are authenticated as: anonymous
Groups that you are in:
  anonymous
Permission you need to have (but didn't): hudson.model.Hudson.Read
 ... which is implied by: hudson.security.Permission.GenericRead
 ... which is implied by: hudson.model.Hudson.Administer
-->

</body></html>


        <html><head><meta http-equiv='refresh' content='1;url=/login?from=%2F'/><script id='redirect'
data-redirect-url='/login?from=%2F' src='/static/7d3df06c/scripts/redirect.js'></script></head><body s
tyle='background-color:white; color:white;'>
Authentication required
<!--
You are authenticated as: anonymous
Groups that you are in:
  anonymous
Permission you need to have (but didn't): hudson.model.Hudson.Read
 ... which is implied by: hudson.security.Permission.GenericRead
 ... which is implied by: hudson.model.Hudson.Administer
-->

</body></html>


        <html><head><meta http-equiv='refresh' content='1;url=/login?from=%2F'/><script id='redirect'
data-redirect-url='/login?from=%2F' src='/static/7d3df06c/scripts/redirect.js'></script></head><body s
tyle='background-color:white; color:white;'>
Authentication required
<!--
You are authenticated as: anonymous
Groups that you are in:
  anonymous
Permission you need to have (but didn't): hudson.model.Hudson.Read
 ... which is implied by: hudson.security.Permission.GenericRead
 ... which is implied by: hudson.model.Hudson.Administer
-->

</body></html>
```

Figure 8 — All resources destroyed.

Result

Terraform and Docker were installed. Configuration files defining a network, a variable number of nginx web containers and a load balancer were written, then initialized, planned and applied to provision the infrastructure. The deployment was verified through the load balancer and then scaled from two to four web servers by changing a single variable. Task 2 is complete.